Visualising Feature Learning in Convolutional Neural Networks

Name	Student ID	GitHub
Olasupo Abimbola	**24115208**	**Repository Link**

1. Introduction

Convolutional Neural Networks (CNNs) have become one of the most important models in modern machine learning and computer vision,it undergoes tasks such as image classification, object detection, and medical imaging. The success of CNN can be largely attributed to it being able to automatically learn useful representations from raw image data, removing the need for traditional feature engineering.

It is however daunting to find out that CNNs are treated as a “black box”, where it is unclear what the model has actually learned internally. This lack of interpretability raises important questions: What features does the network focus on? How does it transform raw pixel data into meaningful predictions? And how does this process change depending on the complexity of the input data?

One approach to answering these questions is through **feature map visualisation**. Previous research, such as the work by Zeiler and Fergus (2014), has shown that early layers in CNNs tend to learn simple features such as edges and textures, while deeper layers capture more complex and abstract representations.

This tutorial builds on these ideas by exploring how feature learning differs when applied to datasets of varying complexity. Specifically, it compares:

- ➔ **MNIST**, a simple grayscale dataset of handwritten digits of 28 by 28 pixel
- ➔ **CIFAR-10**, a more complex dataset of colour images (RGB) representing real-world objects OF 32 BY 32 pixel

2. Objective

The objective of this tutorial is to investigate how Convolutional Neural Networks learn internal feature representations and how these representations evolve across layers.

Rather than focusing solely on classification performance, this work aims to understand the internal behaviour of CNNs by visualising feature maps. In particular, it seeks to answer the following questions:

- ➔ What types of features are learned in early versus deeper convolutional layers?
- ➔ How does feature complexity evolve as data passes through the network?
- ➔ How does the nature of learned features differ between simple and complex datasets?

3. How CNNs Learn Features

A CNN takes an image as input and applies a series of operations that transform the raw pixel values into increasingly meaningful representations. The key components involved in this process are convolution, activation functions, and pooling.

3.1. Convolution

A convolutional layer applies a set of small filters of a typical size, 3×3, to which learns to detect precise patterns in the input These filters are also known as kernels, and they slide across the image detecting patterns.

For example, some filters may respond strongly to:

- ➔ edges
- ➔ lines
- ➔ gradients in intensity

3.2. Hierarchical Feature Learning

One of the most important properties of CNNs is **hierarchical learning**.

- ➔ **Early layers** acts directly on raw pixel values and typically learns simple features such as edges, lines, curves etc.
- ➔ **Intermediate layers** combine these from the early layers into more complex patterns.
- ➔ **Deeper layers** learn abstract representations that are useful for classification.

3.3. Activation Functions

An activation function such as ReLU (Rectified Linear Unit), sigmoid… is applied, after convolution layer, and this introduces non-linearity, allowing the network to learn more complex relationships.

3.4. Pooling

Pooling layers essentially reduce the spatial size of the feature maps. In this tutorial, max pooling selects the maximum value from minor regions of the feature map.

```
max_pool = nn.MaxPool2d(kernel_size=2, stride=2)
```

3.5. From Features to Predictions

After several convolutional and pooling layers, the feature maps are flattened into a one-dimensional vector and passed through dense (fully connected) layers.

These layers combine the learned features to produce final predictions class probabilities.

3.6. Why Feature Visualisation Matters

Although CNNs are highly effective, their internal operations are not always intuitive. Feature map visualisation provides a way to observe what the network is learning at different stages.

By examining feature maps, we can:

- ➔ Identify what patterns the network responds to
- ➔ understand how representations evolve across layers
- ➔ Compare how different datasets influence feature learning

4. Methodology

In order to investigate feature learning in CNNs, a controlled experimental setup was used. The same CNN architecture was applied to two datasets of varying complexity, allowing for a direct comparison of learned feature representations.

5. Datasets

Two datasets were selected to represent different levels of visual complexity.

5.1. MNIST

MNIST, as in *Figure 1*, is a grayscale dataset of handwritten digits ranging from 0 to 9 with each image having a resolution of 28×28 pixels and a single channel.

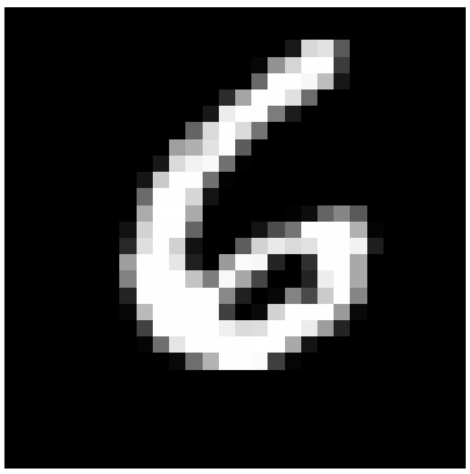

Figure 1: MNIST Training Data (Digit 6)

5.2. CIFAR-10

CIFAR-10, as in *Figure 2*, is a dataset consisting of 32×32 colour images belonging to 10 different classes, including animals and vehicles.

Figure 2: CIFAR Training Images

6. Preprocessing

Before training the CNN, both datasets were preprocessed to ensure compatibility with the model and to improve training performance.

6.1. Normalisation

Pixel values in both datasets originally range from 0 to 255. These values were scaled to the range 0 to 1 by dividing by 255.

This step is important because:

- ➔ It stabilises training
- ➔ It prevents large input values from dominating learning
- ➔ It helps the optimiser converge more efficiently

6.2. Reshaping

MNIST images are initially stored as 2D arrays of shape (28, 28). However, CNNs expect input in the form (height, width, channels).

Therefore, MNIST images were reshaped to include a channel dimension, resulting in a shape of (28, 28, 1).

CIFAR-10 images already include three colour channels (32, 32, 3), so no reshaping was required.

7. CNN Architecture

A simple Convolutional Neural Network was designed to prioritise interpretability over complexity.

The architecture consists of the following layers as seen in *Table 1* and *Table 2*, respectively.

Model: "sequential_9"

Layer (type)	Output Shape	Param #
conv2d_18 (Conv2D)	(None, 30, 30, 16)	448
max_pooling2d_18 (MaxPooling2D)	(None, 15, 15, 16)	0
conv2d_19 (Conv2D)	(None, 13, 13, 32)	4,640
max_pooling2d_19 (MaxPooling2D)	(None, 6, 6, 32)	0
flatten_9 (Flatten)	(None, 1152)	0
dense_18 (Dense)	(None, 64)	73,792
dense_19 (Dense)	(None, 10)	650

Total params: 79,530 (310.66 KB)
Trainable params: 79,530 (310.66 KB)
Non-trainable params: 0 (0.00 B)

Table 1: Summary of CNN Architectures (CIFAR-10).

Model: "sequential_8"

Layer (type)	Output Shape	Param #
conv2d_16 (Conv2D)	(None, 26, 26, 16)	160
max_pooling2d_16 (MaxPooling2D)	(None, 13, 13, 16)	0
conv2d_17 (Conv2D)	(None, 11, 11, 32)	4,640
max_pooling2d_17 (MaxPooling2D)	(None, 5, 5, 32)	0
flatten_8 (Flatten)	(None, 800)	0
dense_16 (Dense)	(None, 64)	51,264
dense_17 (Dense)	(None, 10)	650

Total params: 56,714 (221.54 KB)
Trainable params: 56,714 (221.54 KB)
Non-trainable params: 0 (0.00 B)

Table 2: Summary of CNN Architectures (MNIST).

Design Rationale

The model was intentionally kept relatively shallow for the following reasons:

- ➔ A simpler architecture makes feature maps easier to interpret
- ➔ Deeper networks produce highly abstract representations that are difficult to visualise
- ➔ The focus of this work is on understanding feature learning, not achieving maximum accuracy

8. Training and Model Behaviour

Using the same architecture and training settings, the model was trained separately on MNIST and CIFAR-10 dataset. This allows us compare directly how the model learns under different levels of data complexity.

8.1. Training on MNIST

```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 8s 7ms/step - accuracy: 0.9337 - loss: 0.2227 - val_accuracy: 0.9792 - val_loss: 0.0724
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9790 - loss: 0.0672 - val_accuracy: 0.9868 - val_loss: 0.0453
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9844 - loss: 0.0492 - val_accuracy: 0.9858 - val_loss: 0.0487
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9890 - loss: 0.0371 - val_accuracy: 0.9887 - val_loss: 0.0448
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9904 - loss: 0.0301 - val_accuracy: 0.9898 - val_loss: 0.0432
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9919 - loss: 0.0259 - val_accuracy: 0.9895 - val_loss: 0.0388
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9930 - loss: 0.0218 - val_accuracy: 0.9897 - val_loss: 0.0405
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9940 - loss: 0.0181 - val_accuracy: 0.9887 - val_loss: 0.0487
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9955 - loss: 0.0137 - val_accuracy: 0.9863 - val_loss: 0.0548
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9959 - loss: 0.0131 - val_accuracy: 0.9897 - val_loss: 0.0432
```

Table 3: Epoch-wise Model Performance (MNIST).

In *Table 3*, the model achieved very high accuracy (approximately 99% on the test set) which shows that the network successfully learned the patterns required to distinguish between several handwritten digits.

This high performance ccan be attributed to the simplicity of the dataset..

➔ digits are well organised	➔ Very little background noise	➔ Obvious visual differences between classes

8.2. Training on CIFAR-10

In contrast, when the same model was applied to CIFAR-10 in *Table 4*, the accuracy was significantly lower (approximately 65–70%).

```
Epoch 1/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.4227 - loss: 1.5977 - val_accuracy: 0.5188 - val_loss: 1.3617
Epoch 2/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.5393 - loss: 1.3016 - val_accuracy: 0.5722 - val_loss: 1.2229
Epoch 3/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.5843 - loss: 1.1797 - val_accuracy: 0.5992 - val_loss: 1.1492
Epoch 4/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.6165 - loss: 1.0959 - val_accuracy: 0.6226 - val_loss: 1.0740
Epoch 5/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.6367 - loss: 1.0400 - val_accuracy: 0.6278 - val_loss: 1.0803
Epoch 6/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.6541 - loss: 0.9885 - val_accuracy: 0.6336 - val_loss: 1.0364
Epoch 7/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.6708 - loss: 0.9451 - val_accuracy: 0.6296 - val_loss: 1.0408
Epoch 8/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.6849 - loss: 0.9082 - val_accuracy: 0.6538 - val_loss: 1.0024
Epoch 9/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.6963 - loss: 0.8745 - val_accuracy: 0.6550 - val_loss: 0.9907
Epoch 10/10
704/704 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.7082 - loss: 0.8422 - val_accuracy: 0.6686 - val_loss: 0.9536
```

Table 4: Epoch-wise Model Performance (CIFAR-10).

This difference reflects the dataset's increased complexity. CIFAR-10 images:

➔ contain colour information	➔ include varied textures and backgrounds	➔ have less clearly defined object boundaries

Accuracy measures how often the model predicts the correct class, while loss measures how confident and precise those predictions are.

Although MNIST achieved both high accuracy and low loss, CIFAR-10 exhibited lower accuracy and higher loss. This suggests that the model is less confident and less consistent in its predictions on more complex data.

The difference in performance between MNIST and CIFAR-10 highlights an important point: the complexity of the input data directly affects how easily a CNN can learn useful features. This sets the foundation for analysing how feature representations differ across datasets.

9. Feature Map Extraction

9.1. What Are Feature Maps?

A feature map is the result of applying a convolutional filter to an input image. Each filter produces one feature map, that highlights where a specific pattern is detected.

9.2. Why Extract Feature Maps?

Feature maps allow us to observe:

➔ Which parts of the image activate certain filters	➔ How patterns are transformed across layers	➔ How feature complexity increases with depth

This provides insight into the internal workings of the CNN, which would otherwise remain hidden.

9.3. Layer Selection

The first convolutional layer was selected to observe basic feature detection, while the second convolutional layer was selected to observe how these features are combined into more complex representations.

By comparing these two layers, it is possible to visualise the hierarchical nature of feature learning.

10. Results and Visualisation

Feature maps were visualised for both MNIST and CIFAR-10 using sample input images. These visualisations provide direct insight into how the CNN processes different types of data.

10.1. MNIST Feature Maps

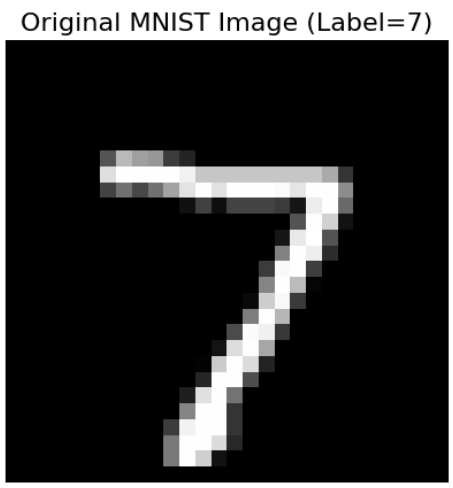

Figure 3: MNIST input image - 7.

For MNIST, the feature maps from the first convolutional layer clearly highlight simple structures such as edges, strokes, and curves. We can observe from *Figure 4*, that many of the filters respond strongly to the digit's boundaries, making the patterns easy to interpret.

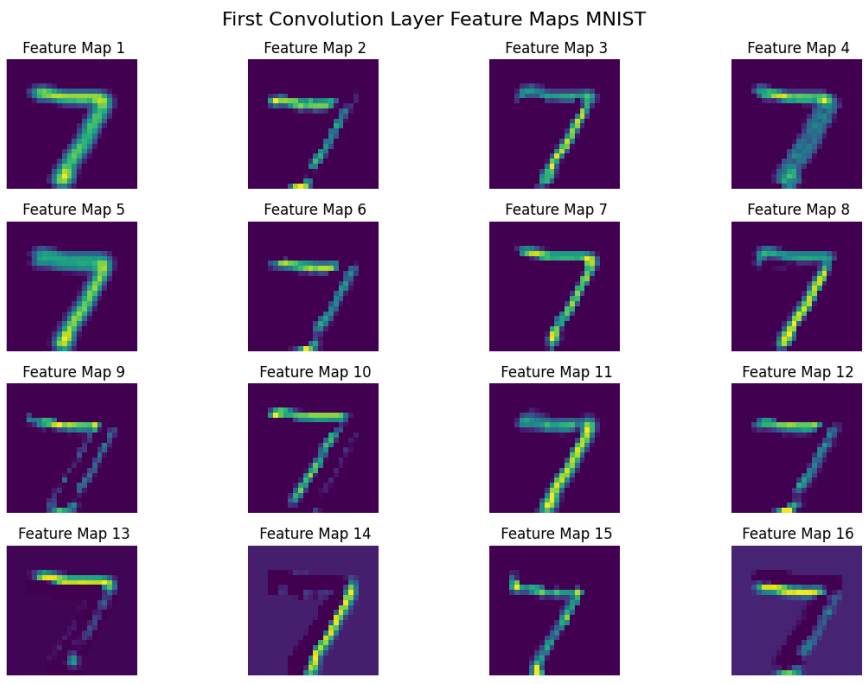

Figure 4: First convolutional layer feature maps (MNIST).

In *Figure 5,* the second convolutional layer, the feature maps become more abstract. Rather than showing simple edges, they represent combinations of features that correspond more closely to parts of digits.

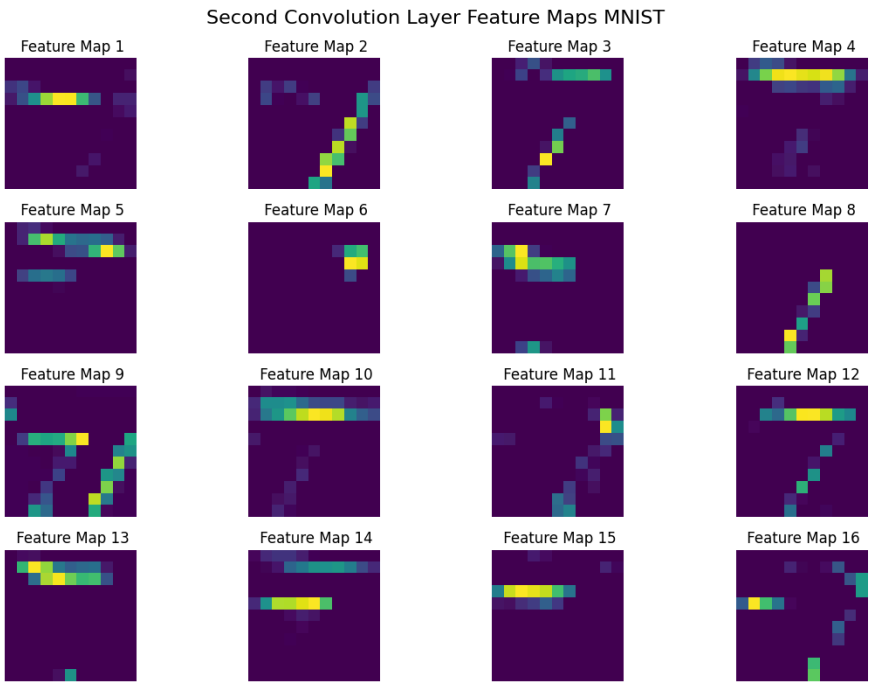

Figure 5: Second convolutional layer feature maps (MNIST).

Overall, the feature maps remain relatively interpretable, reflecting the simplicity of the dataset.

10.2. CIFAR-10 Feature Maps

Figure 6: Sample CIFAR-10 input image (e.g. frog class).

In contrast, as seen in *Figure 7*, the feature maps for CIFAR-10 appear more complex and less visually intuitive.

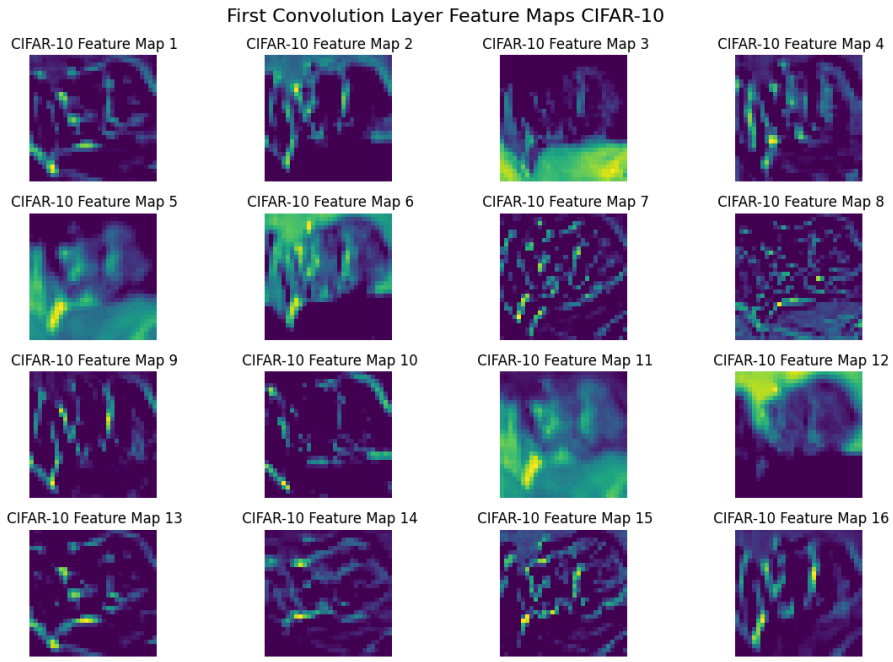

Figure 7: First convolutional layer feature maps(CIFAR-10).

Even in the *Figure 7*, the activations do not clearly correspond to simple shapes. Instead, they capture combinations of colour, texture, and local patterns.

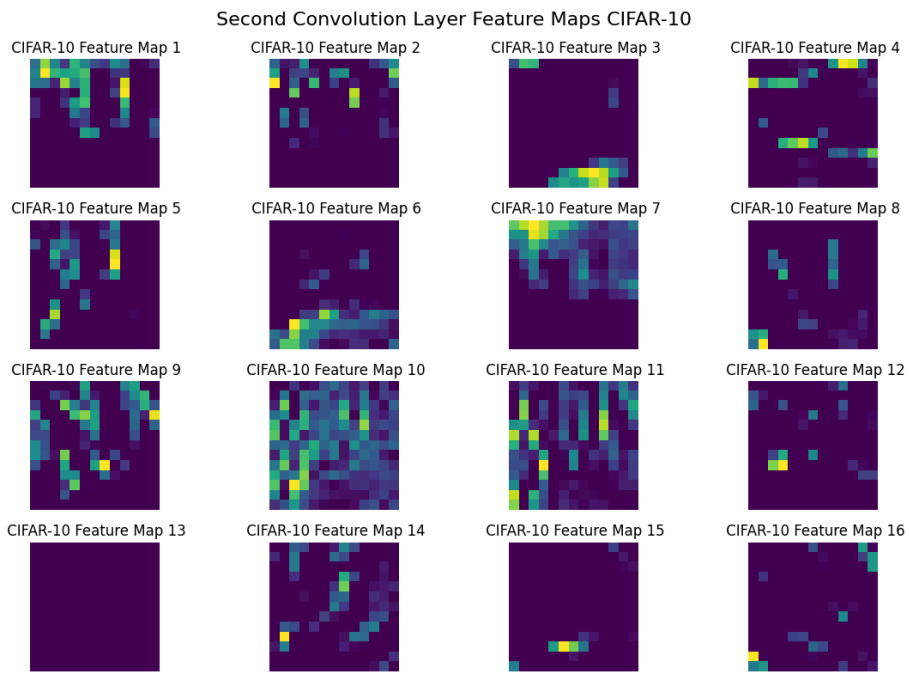

Figure 8: Second convolutional layer feature maps (CIFAR-10).

In *Figure 8*, the second convolutional layer, the feature maps become increasingly abstract. The patterns are more distributed, and it is difficult to associate them with specific object components.

11. Comparison and Discussion

The question, how CNNs learn features under varrying conditions has been greatly solved by the comparison between MNIST and CIFAR-10.

For MNIST, the network detects edges, lines, and curves which are directly recognisable as parts of the digits the which purports that the learned features align closely with human visual perception. This makes the feature maps reasonably easy to interpret.

For CIFAR-10, however, the learned features are more abstract. This is due to several factors:

➔ The presence of RGB colour channels (3)

➔ Increased variability in object appearance

➔ Background noise and texture presence

As a result, the network learns distributed representations that are effective for classification but not easily interpretable by humans.

This highlights an important concept:

As the complexity of input data increases, feature representations become more abstract and less directly aligned with human intuition.

This observation is consistent with findings in previous research on CNN interpretability.

12. Conclusion

In this tutorial we have demonstrated how Convolutional Neural Networks learn hierarchical feature representations from specific image data given complexity and model.

From our analysis we can observe clearly that the early layers capture simple patterns, while deeper layers learn more complex representations by visualiising feature maps across layers.

Our comparison between the MNIST and the CIFAR-10 shows that dataset complexity plays a pivot role in shaping learned features, leading us to learn that simpler datasets produce clearer and more interpretable feature maps, while more complex datasets lead to richer but less intuitive representations, hence requiring more complex models.

Overall, feature map visualisation provides a helpful tool for understanding how CNNs internal computation occurs and helps link model performance with interpretability.

13. References

- A. Krizhevsky, I. Sutskever, and G. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. 2012.
- A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. University of Toronto. 2009.
- Y. LeCun, L. Bottou, Y. Bengio, & P. Haffner. Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE. 1998.
- M. D. Zeiler and R. Fergus, Visualising and Understanding Convolutional Networks. European Conference on Computer Vision (ECCV). 2014.
- F. Chollet, Deep Learning with Python. Manning Publications. 2017.
- TensorFlow. (n.d.). Keras API Documentation.
- Arm. (n.d.). Convolutional Neural Network. What is a Convolutional Neural Network (CNN)?.
- 3Blue1Brown. (n.d.). Neural Network. YouTube Playlist. But what is convolution?. Youtube.
- StatQuest with Josh Starmer. (n.d.). Convolutional Neural Networks (CNNs). Youtube.